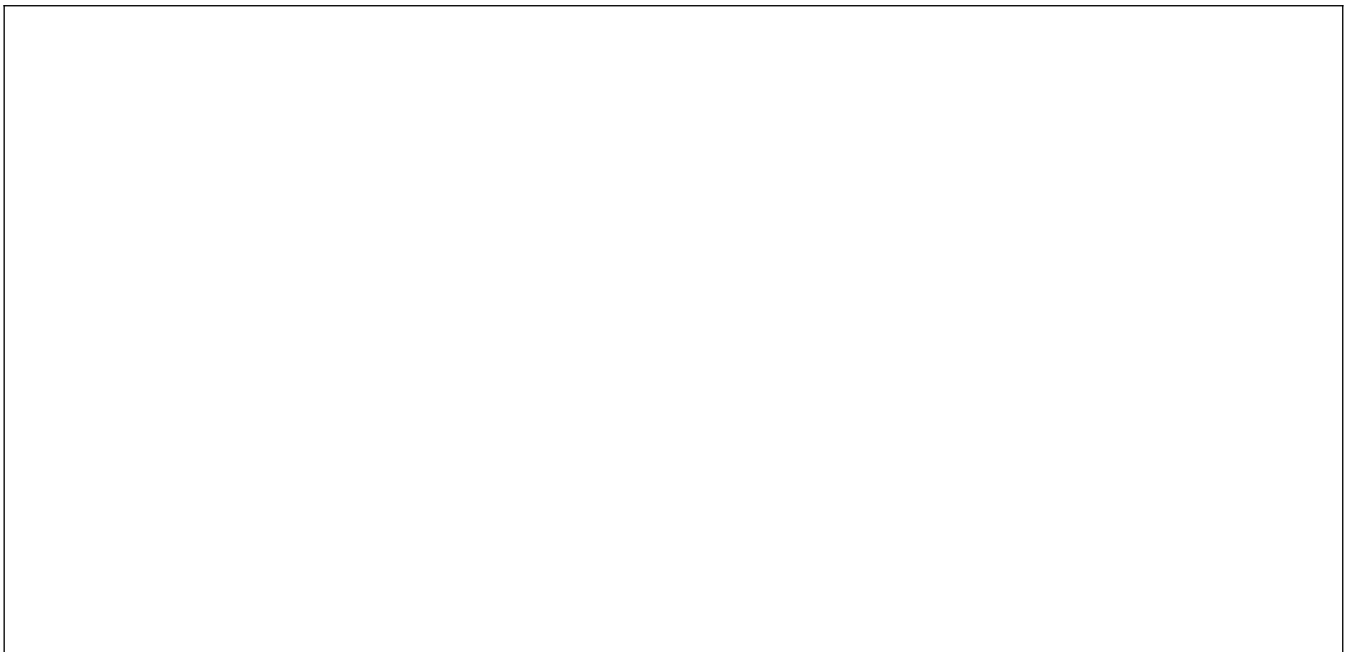


DM NSI

Projet Qui-Est-Ce ?



1. Présentation du projet	<i>p.2</i>
Objectif, architecture, principes de conception	
2. Diagrammes de classes	<i>p.3</i>
Programmation orientée objet : classes Jeu_Affichage	
3. Backlog du produit	<i>p.4</i>
Planification, hiérarchie et revue des fonctionnalités	
4. Carnet de bord du projet	<i>p.5-7</i>
Progression du développement, ajustements, commentaires	

Présentation du projet

L'objectif de ce projet est de concevoir et de développer, en langage Python, une version numérique du jeu « Qui-est-ce ? ». Le programme permet à deux joueurs, ou à un joueur contre un ordinateur, de deviner un personnage en posant des questions basées sur des caractéristiques ou des adjectifs. Pour atteindre cet objectif, nous avons mobilisé les notions de programmation orientée objet (POO), ainsi que les bibliothèques nécessaires à la création d'une interface graphique, notamment la bibliothèque Pygame.

La conception du projet s'est appuyée sur une démarche progressive et structurée, puisque ce programme a été organisé autour de plusieurs composants clairement définis, tels que la gestion de l'affichage du plateau de jeu, le traitement des interactions avec la souris, ainsi que la gestion de l'état du jeu et des actions des joueurs. Cette organisation permet de séparer les différentes responsabilités du programme et de rendre le code plus lisible et plus facile à maintenir.

Le développement a été réalisé étape par étape selon la méthode Agile, en ajoutant progressivement les différentes fonctionnalités du jeu : affichage des personnages, sélection des adjectifs, élimination des personnages incompatibles, gestion des tours de jeu et détermination du vainqueur. Des tests réguliers ont été effectués afin de vérifier le bon fonctionnement du programme à chaque phase du développement.

Enfin, ce projet s'accompagne de la présence d'une documentation claire, d'un code lisible et de tests réguliers, ayant pour objectif de garantir la fiabilité du programme et d'en faciliter sa compréhension, aussi bien pour l'utilisateur que pour un développeur extérieur.

Diagrammes de classes

Classe Jeu_Affichage

→ <code>__init__()</code> : None	→ <code>Adjectif_Click(Boutton: str)</code> : None
→ <code>get_frame(img: Surface, framez: int ou float, nb_colonnes: int, nb_lignes: int)</code> : Surface	→ <code>Get_Last_State_Action()</code> : None
→ <code>charger_image(Data: dict)</code> : Surface	→ <code>Bouton_Retour()</code> : None
→ <code>initialiser_fenetre()</code> : None	→ <code>Get_Character_With_Button(Boutton: str)</code> : Personnage ou None
→ <code>surface_arrondi(surface: Surface, radius: int ou float)</code> : Surface	→ <code>Clickable_Func()</code> : None
→ <code>clamp(n: int ou float, min: int ou float, max: int ou float)</code> : int ou float	→ <code>Remove_Filter(Button: str)</code> : None
→ <code>Ajouter_Filtre(surface: Surface, color: tuple ou list, opacite: int ou float)</code> : None	→ <code>Create_Column_Button(Data: dict)</code> : tuple
→ <code>lerp(a: int ou float, b: int ou float, t: int ou float)</code> : int ou float	→ <code>Create_Column(Data: dict)</code> : None
→ <code>lerp_tuple(data: dict)</code> : tuple	→ <code>Afficher_Action()</code> : None
→ <code>Get_Boost(perso: str)</code> : int ou float	→ <code>Setup_Player(Player: str, Bot: bool)</code> : None
→ <code>Get_Offset(data: dict)</code> : tuple	→ <code>Playing_Bot()</code> : None
→ <code>Add_Boutton(Data: dict)</code> : Surface	→ <code>GameMode(bot: bool)</code> : None
→ <code>Add_Collidable(Surface: Surface, Nom: str, Position: tuple)</code> : None	→ <code>deviner_state()</code> : None
→ <code>Remove_Collidable(Nom: str)</code> : None	→ <code>Winner()</code> : None
→ <code>afficher()</code> : None	→ <code>Remove_Button_Start_Collidable()</code> : None
→ <code>NewState(Next: str)</code> : None	→ <code>start_state()</code> : None
→ <code>Remove_Button(Button: str)</code> : None	→ <code>Afficher_Regle_Du_Jeu()</code> : None
→ <code>Remove_All_Button()</code> : None	→ <code>Afficher_State()</code> : None
→ <code>Click_Start(Boutton: str)</code> : None	→ <code>Reset_Jeu()</code> : None
→ <code>IsIterable(obj: any)</code> : bool	→ <code>Events()</code> : None
→ <code>Have_Adjectif(Data: dict)</code> : bool	→ <code>Transition()</code> : None
→ <code>get_adj(data: dict)</code> : list ou dict	→ <code>Transition_Out()</code> : None
→ <code>Random_Adjectif()</code> : list	→ <code>Background()</code> : None
→ <code>Remove_Adjectif(Adjectif: str)</code> : None	→ <code>Lancer()</code> : None
→ <code>BackToChoice()</code> : None	→ <code>Create_Texte(Data: dict)</code> : Surface
→ <code>Get_Other_Player()</code> : str	→ <code>Closest_Clickable_Func()</code> : str ou None
→ <code>Change_Player()</code> : None	→ <code>Affiche_Message()</code> : None
→ <code>Get_Last_Adjectif()</code> : list ou dict	→ <code>Message_On_Screen(Text: str, Duration: int ou float, Scale: int ou float, Color: tuple, TextSize: int ou float, Outline: bool)</code> : None

Backlog du produit

Fonctionnalités	Priorité	Fait
• <u>Créer une classe modélisant un qui-est-ce</u> • <u>Définir les attributs de chaque personnage précisément</u> • <u>Pouvoir manipuler en format pygame une interface visuelle</u> • <u>Accessibilité par différents modes de jeu</u> • <u>Implémentation de règles claires</u> • <u>Explications concises grâce à la documentation</u> • <u>Permettre à l'utilisateur de rejouer des parties</u>	• Essentielle • Essentielle • Essentielle • Secondaire • Secondaire • Essentielle • Secondaire	✓ ✓ ✓ ✓ ✓ ✓ ✓

Carnet de bord du projet

Date	Fait	Difficultés	A faire
20/02 1h	Backlog/ Attribution des rôles	Déterminer les différentes règles du jeu	Définir les attributs des personnages de la classes / commencer le mode de jeu « Deviner »
28/02 1h	Mise en place de la Classe Jeu_Affichage : Charger_image() ; Initialiser_fenetre() ; Afficher_State() ; Lancer() ; Setup_Player()	Optimiser l'affichage de nombreuse images chaque image par seconde	Améliorer l'interface graphique avec des boutons
01/02 2h30	Amélioration de l'interface graphique avec des boutons, un menu, un fond d'écran, des transitions et en arrondissant les contours des images Create_Column_Button() Afficher_Action() ; Create_Texte() ; Create_Column() ; Events() ; Background() ; Transition() ; Transition_Out() ; surface_arrondi() ; clamp() ; Ajouter_Filtre() ; lerp() ; lerp_tuple() ; get_Boost() ; get_Offset() ; Add_Boutton() ; Add_Collidable() ; Remove_Collidable() ; NewState() ; Remove_Button() ; Remove_All_Button() ; Click_Start() ; Clickable_Func() ; deviner_state() ; start_state() ; Remove_Filter()	Trouver une cohérence dans la direction artistique du jeu mettre en place une interpolation linéaire pour l'animation des boutons Changer l'état du jeu après 1 seconde mettre en pause la boucle, possible grâce au module «threading »	Mettre en place algorithme de recherche d'attributs des personnages / éliminer les personnages qui ne correspondent pas aux attributs du personnage à deviner

02/03 2h	Mise en place de l'algorithme de recherche des attributs par personnage possibilité de revenir au menu ou au choix précédent d'attributs si les attributs des personnages ne correspondent au personnage à deviner alors ils sont éliminés avec un filtre noir <pre> Remove_Adjectif() ; Get_Last_Adjectif() ; Adjectif_Click() ; Come_Back_Menu() ; Get_Last_State_Action() ; BackToChoice() ; Bouton_Retour() ; Closest_Clickable_Func() ; Reset_Jeu() ; Is_Iterable() ; Have_Adjectif() ; get_adj() </pre>	Algorithme de recherche des attributs difficile à cause du système d'arbre avec différents choix	Améliorer l'interaction entre les joueurs et le jeu en mettant en place des messages sur l'écran
03/03 1h	Amélioration de l'interaction entre les joueurs et le jeu en mettant en place des messages sur l'écran du choix d'un attribut, indiquant si le joueur est sur la bonne piste ou non. <pre> Remove_Button_Start_Collidable() ; Message_On_Screen() ; Affiche_Message() ; get_frame() </pre>	Faire une image animé grâce au flipbook	Faire un affichage du gagnant avec son score ; afficher les joueurs avec des contours jaune pour faire apparaître explicitement leurs attributs en passant la souris sur les boutons ; faire mode de jeu 1v1 contre ordinateur et mode de jeu 1v1 avec un autre joueur ; pouvoir rejouer au jeu une fois une partie terminée

04/03 1h30	Affichage du gagnant avec son score : Si le joueur arrive a trouver le personnage a deviner avec le moins de questions possible, alors plus son score sera élevé Afficher les personnages avec des contours jaune pour faire apparaître explicitement leurs attributs en passant la souris sur les boutons Création du mode de jeu 1vs1 contre ordinateur et mode de jeu 1vs1 avec une personne réelle Tour par tour, le premier qui trouve le personnage a deviner de l'autre l'emporte Ajout de la possibilité de rejouer au jeu une fois une partie terminée Playing_Bot() ; GameMode() ; Winner() ; Change_Player() ; Get_Other_Player() ; Random_Adjectif() ;	Permettre à l'ordinateur de choisir aléatoirement un attribut sur lequel poser une question	Faire un 4eme onglet « Règle du jeu »
05/02 1h	Création et mise en place d'une fiche permettant de comprendre les différents mode de jeu : -1vs1 JOUEUR -1vs1 BOT -Deviner Comprendre le système global des points et des attributs Afficher_Regle_Du_Jeu()	Écriture des règles sur une fiche correspondant à la direction artistique du jeu	Finaliser la documentation Relecture finale du projet et mise en forme du PDF
06/03 1h	Tests du programme, ajout de la documentation et finition du PDF	Aucune difficulté majeure	